Vivekanand Education Society's
Institute of Technology
(Autonomous Institute Affiliated to University of Mumbai, Approved by AICTE & Recognised by Govt. of Maharashtra)
NAAC accredited with 'A' grade
Department of Information Technology

FSD MINI-PROJECT REPORT

ON

CHATFLOW: REAL-TIME COMMUNICATION AND COLLABORATION PLATFORM FOR SOFTWARE DEVELOPMENT TEAMS

Submitted in partial fulfilment of the requirements for the

Full Stack Development (FSD) Mini-Project

Semester 5 | Academic Year 2026-27

Submitted by

Krrish Gupta

Class D15B | B.Tech – Information Technology

Project Guide: **Mrs. Pooja Prajapati**

Vivekanand Education Society's
Institute of Technology
(Autonomous Institute Affiliated to University of Mumbai, Approved by AICTE & Recognised by Govt. of Maharashtra)
NAAC accredited with 'A' grade
Department of Information Technology

Declaration

I, Krrish Gupta, Class D15B, student of Semester 5, Third year Engineering from Department of Information Technology, V. E. S. Institute of Technology, hereby declare that the project titled ChatFlow: Real-Time Communication and Collaboration Platform for Software Development Teams, submitted to the University of Mumbai for the academic year 2026-27, is a record of an original work done by me under the guidance of Mrs. Pooja Prajapati, Assistant Professor, Department of Information Technology, V.E.S. Institute of Technology, Mumbai -400074.

This project report is submitted as a partial fulfillment of the requirement of the subject Full Stack Development in Semester 5.

Date: _____

Place: Mumbai

Name: Krrish Gupta

Signature: _____

Vivekanand Education Society's
Institute of Technology
(Autonomous Institute Affiliated to University of Mumbai, Approved by AICTE & Recognised by Govt. of Maharashtra)
NAAC accredited with 'A' grade
Department of Information Technology

CERTIFICATE

This is to certify that the Mini Project Report entitled ChatFlow: Real-Time Communication and Collaboration Platform for Software Development Teams submitted by Mr. Krrish Gupta, of Class D15B, Department of Information Technology, V.E.S. Institute of Technology, Mumbai, in partial fulfillment of the requirements for the subject Full Stack Development under the University of Mumbai, is a bonafide record of work carried out by the student under my guidance and supervision during the Academic Year 2026-27.

The work presented in this report is original and has been completed as per the prescribed academic requirements.

Project Guide:

Name: Mrs. Pooja Deepak Prajapati

Signature: _____

Head of Department:

Department of Information Technology

Signature: _____

ABSTRACT

ChatFlow is a full-stack real-time communication and collaboration platform architected specifically for software development teams, tech startups, open-source maintainers, and student engineering cohorts. While conventional instant messaging applications lack engineering workflow context, ChatFlow integrates developer-centric collaboration directly into channels and direct conversations. The frontend is built using React 19, Vite, Tailwind CSS, Lucide React icons, and React Router DOM; user authentication, active workspaces, channels, online presence, and real-time state synchronization are centrally managed using the React Context API and specialized hooks (useChatState and useDebounce).

The backend is engineered with Node.js (ES Modules) and Express 4, providing RESTful endpoints for user registration, login, multi-tenant workspace administration, workspace-scoped public and private channels, direct 1-to-1 chats, GitHub integrations, bug report management, and CI/CD deployment streaming. MongoDB Atlas serves as the persistent cloud database through Mongoose schemas. Passwords are securely hashed with bcrypt (10 salt rounds), REST endpoints and Socket.IO connections are safeguarded by JSON Web Tokens (JWT) and role-based access control (Admin, Member), while Helmet, CORS, input validation, and request rate limiting harden API resilience.

Socket.IO enables low-latency bi-directional communication, powering workspace broadcasts, channel rooms, direct chats, typing indicators, active presence, message threads, inline editing, and read status delivery receipts. ChatFlow introduces a cryptographically verified GitHub webhook integration utilizing HMAC SHA-256 signatures (X-Hub-Signature-256) to automatically post rich interactive Pull Request cards and Git commit activities into dedicated channels. Discussion messages can be converted into tracked bug reports with severity and status filters in a single click, alongside live CI/CD deployment pipeline feeds. Frontend and backend services are containerized via Docker and Docker Compose with Nginx as a reverse proxy, verified through GitHub Actions CI/CD workflows, and deployed to Vercel (frontend) and Render (backend).

Comprehensive verification using Postman, curl, Docker, and multi-user browser testing validated account authentication, role-based authorization, real-time message delivery, thread replies, dynamic unread badge clearing, webhook security, and cloud deployment resilience.

Keywords: Full-stack development, React, Express, MongoDB Atlas, Socket.IO, JWT, GitHub Webhooks, HMAC SHA-256, Docker, GitHub Actions, Vercel, Render.

TABLE OF CONTENTS

Topic / Chapter	Page No.
Declaration	2
Certificate	3
Abstract	4
1. Introduction and Objectives	6
2. Project Scope and Requirements	7
3. Technologies Used	8
4. System Design	9
5. Implementation	12
6. Implementation with Screenshots	16
7. Testing and Results	26
8. Deployment and DevOps	28
9. Conclusion and Future Scope	30
10. References	31
Project Links Summary	32

1. INTRODUCTION AND OBJECTIVES

1.1 Introduction

ChatFlow is a real-time web-based communication and team collaboration platform built as a Full Stack Development (FSD) mini-project. It combines a modern React 19 frontend with a robust Node.js/Express backend, a MongoDB Atlas cloud database, and a high-performance Socket.IO real-time engine. ChatFlow provides team workspace creation, public and private channels, direct 1-to-1 conversations, GitHub Pull Request & commit integration cards, message-to-bug conversion, CI/CD deployment activity streaming, and dynamic unread message tracking.

Modern software engineering teams frequently experience communication fragmentation across disparate tools: chat in Slack or Discord, code reviews on GitHub, bug tracking in Jira, and deployment monitoring in cloud consoles. ChatFlow consolidates these engineering workflows into a cohesive, high-performance real-time hub tailored for student development teams, startups, and open-source contributors.

1.2 Problem Statement

Generic chat systems lack tight developer workflow integrations. When engineers discuss issues, pull requests, and production releases, they are forced to manually copy-paste external URLs, context-switch between browser tabs, and lose conversational records. Furthermore, building such a collaborative system requires cleanly separating presentation from API logic, protecting multi-tenant workspace resources, enforcing cryptographic signature verification on external webhooks, securely hashing credentials, and synchronizing unread message counters across active client sessions.

1.3 Aim

To design, build, test, containerize, and deploy a secure, responsive full-stack team collaboration platform with JWT authentication, multi-tenant workspaces, persistent real-time channels and direct messaging, automated GitHub webhook verification, bug tracking, and continuous integration/continuous deployment (CI/CD).

1.4 Objectives

- **UI/UX:** Build an intuitive, responsive developer interface using React 19, Vite, Tailwind CSS, and Lucide React with client-side routing.
- **State Management:** Manage complex asynchronous state, presence, and real-time Socket.IO subscriptions via React Context API and custom hooks.
- **REST APIs:** Implement modular Express REST APIs backed by Mongoose schemas for users, workspaces, channels, conversations, messages, repositories, bug reports, and deployments.
- **Security:** Enforce bcrypt password hashing, JWT bearer token validation, role-based authorization (Admin/Member), input validation, and request rate limiting.

- **Webhooks & HMAC:** Integrate GitHub webhooks with HMAC SHA-256 signature verification (X-Hub-Signature-256) to ingest PR and commit events in real time.
- **Real-Time Messaging:** Implement Socket.IO channels, direct messaging, typing indicators, read receipts, and message threads stored persistently in MongoDB.
- **Containerization:** Containerize the frontend and backend using Docker and Docker Compose with an Nginx reverse proxy.
- **DevOps & Cloud:** Automate CI/CD with GitHub Actions workflows and deploy frontend to Vercel, backend to Render, and database to MongoDB Atlas.

1.5 Project Links

Resource	Platform	Link
Source code	GitHub	https://github.com/Krrish-Gupta-18/fs-project
Frontend (live)	Vercel	https://fs-project-theta.vercel.app/
Backend API (live)	Render	https://fs-project-backend.onrender.com
Database	MongoDB Atlas	Cloud-hosted cluster (chatflow database)

2. PROJECT SCOPE AND REQUIREMENTS

2.1 Project Scope

ChatFlow is a fully functional full-stack developer communication platform. It encompasses client-side presentation, RESTful API architecture, cloud database modeling, authenticated WebSockets, external webhook ingestion with cryptographic signature validation, bug report tracking, containerized local execution, and cloud hosting. The scope also includes automated CI/CD pipelines on GitHub Actions ensuring quality validation before deployment.

2.2 Functional Requirements

- **Authentication & Profiles:** Secure user registration and login with bcrypt password hashing, JWT tokens, avatar selection, and profile bio inspection.
- **Multi-Workspace System:** Create and switch workspaces, manage role-based memberships (Admin/Member), and generate shareable alphanumeric invite codes.
- **Channels & Direct Chats:** Support public channels (#general, #development, #pull-requests) and private channels scoped to workspaces, alongside direct 1-to-1 conversations.

- **Rich Messaging:** Code block formatting with syntax styling, @mentions, inline message editing, deletion, image/file attachments, and threaded replies.
- **Developer Activity Shortcuts:** One-click sidebar access to bug trackers, repository activities, and deployment release streams.
- **GitHub HMAC Webhooks:** Receive pull request and push webhook events verified via HMAC SHA-256 and render interactive preview cards.
- **Message-to-Bug Conversion:** Convert any chat message into a tracked bug report with severity levels (Critical, High, Medium, Low) and assignee tracking.
- **Dynamic Unread Counters:** Accurately track and display unread badges on direct chats and channels, clearing dynamically when viewed.

2.3 Non-Functional Requirements

Requirement	Project approach
Usability	Modern dark/light developer aesthetic, sidebar navigation, keyboard shortcuts, unread badges, and clear responsive layouts.
Maintainability	Modular architecture separating controllers, models, routes, hooks, components, and socket handlers.
Security	Bcrypt password hashing (10 rounds), JWT authentication, role checks, raw body HMAC SHA-256 verification, Helmet, and rate limiting.
Portability	Dockerfiles, Docker Compose, and Nginx reverse-proxy configuration define an identical reproducible runtime across platforms.
Availability	Deployed to high-availability cloud infrastructure on Vercel, Render, and MongoDB Atlas with automated health checks.
Real-time interaction	Socket.IO delivers messages, typing status, presence, and read receipts instantaneously with zero page reloads.

3. TECHNOLOGIES USED

ChatFlow employs an end-to-end JavaScript/ESM software stack, a cloud document database, event-driven WebSocket communication, and modern DevOps tools. The table below outlines the core technologies and their respective roles.

Technology / Tool	Role in ChatFlow
React 19 + React Router	Component-based single-page application (SPA) and client-side routing between workspaces, channels, and chats.
Vite 6	Fast frontend build tooling, HMR development server, and optimized production bundling.

Technology / Tool	Role in ChatFlow
Tailwind CSS 3	Utility-first styling for dark-mode developer UI, responsive layouts, badges, and modal drawers.
Lucide React	Consistent, developer-friendly vector iconography for channels, repos, bugs, and status indicators.
Hooks / Context API	Global state management (useChat, useChatState, useDebounce, AuthContext) for reactive data.
Node.js 24 + Express 4	Backend REST API, webhook signature verification, route middleware, and HTTP server.
MongoDB Atlas + Mongoose 8	Cloud NoSQL document database storing users, workspaces, channels, messages, bugs, and deployments.
bcryptjs + JWT	Cryptographic password hashing and JSON Web Token issuance and verification.
Socket.IO 4	Full-duplex WebSocket communication for real-time messaging, typing indicators, and presence.
Docker + Docker Compose	Containerization of frontend and backend services for reproducible local and server execution.
Nginx	High-performance reverse proxy routing HTTP API and WebSocket connections to backend.
GitHub Actions	Automated CI/CD pipelines executing build validation and triggering deployments to Vercel and Render.
Vercel, Render, Atlas	Serverless cloud hosting for frontend SPA, backend Web Service, and database cluster.

3.1 Development and Deployment Environment

Development was conducted in Visual Studio Code with Node.js 24 and npm. The Vite client runs on port 3000 (and 5173 during standalone development), the Express backend listens on port 5000, and Docker Compose publishes frontend on port 3000 and backend on port 5000. Docker images are built from node:22-alpine and nginx:alpine. Production deployments are orchestrated automatically via GitHub Actions: Vercel serves the frontend SPA with SPA rewrite rules in vercel.json, Render hosts the Node.js backend web service, and MongoDB Atlas manages the database cluster.

4. SYSTEM DESIGN

ChatFlow employs a layered client-server architecture with dual communication channels. The React SPA sends standard HTTP REST requests for CRUD operations and establishes a persistent Socket.IO connection for real-time messaging, presence, and webhook notification broadcasts. Figure 1 illustrates the high-level architecture.

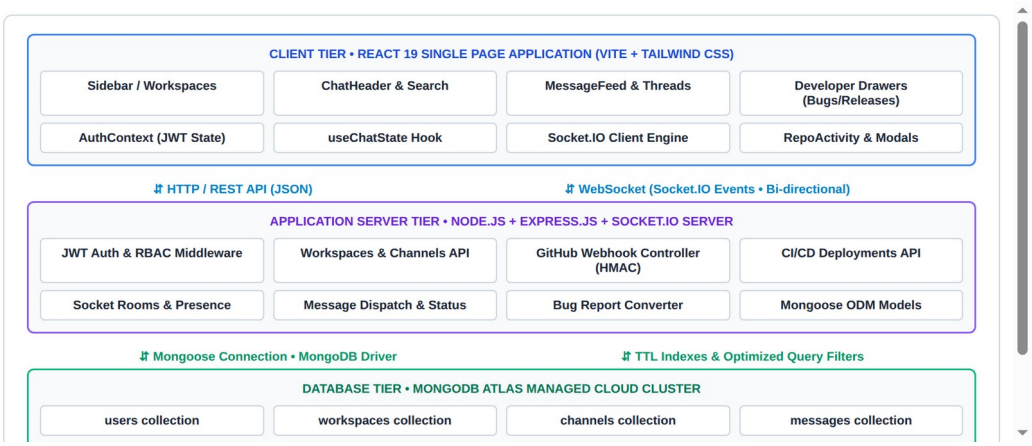


Figure 1. High-level architecture of ChatFlow (React SPA, Express + Socket.IO server, MongoDB Atlas).

4.1 Architecture Components

Component	Responsibility
Browser / Client	Renders React components, manages UI themes, handles local state, sends REST requests, and maintains Socket.IO connection.
Frontend Service	Vite-bundled React application including Sidebar, ChatHeader, MessageBubble, MessageInput, ThreadView, BugTracker, and modals.
Backend Service	Express.js server handling auth, workspaces, channels, messages, GitHub webhooks, bug reports, and Socket.IO events.
MongoDB Atlas	Managed cloud database storing users, workspaces, channels, messages, repositories, bug reports, and deployments.
Nginx Reverse Proxy	Serves frontend production build, falls back to index.html for SPA routes, and proxies /api/ and /socket.io/ traffic.
GitHub Actions CI/CD	Automated workflows running syntax/build validation on PRs and triggering production deployments on push.
Vercel / Render	Hosting platforms providing global CDN distribution for the frontend and container runtime for the backend.

4.2 System Workflow and Project Structure

4.2.1 Main Request Flow

- **1:** The user accesses ChatFlow; the client checks localStorage for an auth token and user profile, redirecting to /login if unauthenticated.
- **2:** On login/sign-up, the client sends credentials to /api/auth/login or /api/auth/register; the backend validates input, hashes/verifies password with bcrypt, and returns a signed JWT.
- **3:** The client stores the JWT and establishes an authenticated Socket.IO connection, joining the personal user room and active workspace room.
- **4:** Workspaces, channels, direct conversations, bug reports, and active deployments are loaded via parallel REST queries.
- **5:** Sending a message emits message:send over Socket.IO; the server persists it to MongoDB, updates conversation summaries, and broadcasts message:new to participants.
- **6:** External GitHub webhook events POST to /api/integrations/github/webhook; the backend verifies HMAC SHA-256 signatures and broadcasts cards to #pull-requests.
- **7:** Pushing to main branch triggers GitHub Actions workflows to validate builds and trigger automatic redeployments on Render and Vercel.

4.2.2 Database Design

Collection	Main fields	Purpose
users	userId, name, email, password, avatar, role, status, bio, timestamps	User accounts, authentication credentials, and presence.
workspaces	workspaceId, name, description, ownerId, members[], inviteCode, timestamps	Multi-tenant team workspaces and memberships.
channels	channelId, workspaceId, name, type (public/private), topic, members[]	Workspace-scoped discussion channels.
conversations	conversationId, name, type (direct/group), participants[], lastMessage, timestamps	1-to-1 direct chats and private conversation metadata.
messages	messageId, conversationId, senderId, text, status, replyTo, threadCount, mentions[]	Chat messages, thread replies, and media attachments.
repositories	repoId, workspaceId, name, fullName, webhookSecret, defaultChannelId	Connected GitHub repositories and webhook settings.
bugreports	bugId, workspaceId, channelId, originalMessageId, title, severity, status	Bug reports created directly from conversation messages.
deployments	deploymentId, workspaceId, environment, status, commitSha, author	CI/CD deployment execution logs and build history.

4.2.3 Main Project Structure

Path	Contents
.github/workflows/	backend-render.yml, frontend-vercel.yml CI/CD pipelines
backend/	server.js, socket.js, config/db.js, controllers/, middleware/, models/, routes/
frontend/src/	main.jsx, App.jsx, index.css, components/, hooks/, services/
frontend/src/components/	auth/, chat/ (Sidebar, ChatHeader, MessageBubble, MessageInput, modals)
frontend/src/hooks/	useChat.jsx, useChatState.js, useDebounce.js
Dockerfile.backend, frontend	Container definitions for Node backend and multi-stage Nginx frontend
docker-compose.yml, nginx.conf	Multi-container orchestration and Nginx reverse proxy configuration
vercel.json, render.yaml	Cloud deployment configuration and SPA rewrite rules

5. IMPLEMENTATION

5.1 Frontend Implementation

The client is built with React 19 and Vite 6. App.jsx configures the BrowserRouter with protected routes and state providers. The application supports full responsive views, modal dialogs for workspace and channel creation, developer drawers for bug reports, releases, and thread panels.

```
<BrowserRouter>
  <Routes>
    <Route path="/login" element={<LoginPage />} />
    <Route path="/register" element={<RegisterPage />} />
    <Route path="/chat" element={<ProtectedRoute><ChatPage /></ProtectedRoute>} />
    <Route path="/chat/:conversationId" element={<ProtectedRoute><ChatPage
/></ProtectedRoute>} />
    <Route path="*" element={<Navigate to="/chat" replace />} />
  </Routes>
</BrowserRouter>
```

5.2 State and Hook Management

State is structured through specialized custom hooks. useChatState encapsulates workspace switching, active channel/direct conversation selection, real-time Socket.IO subscriptions, optimistic message dispatch, and dynamic unread counter calculation. useChat provides consuming components clean access to context data without prop drilling.

```
// Sync active target, clear unread count, and emit read receipts
useEffect(() => {
  if (!activeConversationId) return;
  setConversations(prev => prev.map(c => c.id === activeConversationId ? { ...c,
unreadCount: 0 } : c));
  markMessagesAsReadApi(activeConversationId).catch(() => {});
  socket?.emit("message:read", { conversationId: activeConversationId });
}, [activeConversationId]);
```

5.3 Backend Implementation

The Express backend wraps an HTTP server sharing the same port with Socket.IO. Routes are mounted under /api/auth, /api/workspaces, /api/channels, /api/conversations, /api/messages, /api/integrations, and /api/bugs. Mongoose connects to MongoDB Atlas. Passwords are encrypted with bcrypt, API routes are guarded by JWT protect and authorize middlewares, and external webhook bodies preserve raw buffers for cryptographic signature verification.

5.4 Main Routes and Events

Route / Event	Purpose
POST /api/auth/login, register	User authentication and JWT token generation.
GET, POST /api/workspaces	Workspace creation, role assignments, and member invites.
GET, POST /api/channels	Channel listing and channel creation within active workspace.
GET, POST /api/messages	Fetch message history, send replies, edit, and delete.
POST /api/messages/read/:id	Mark all unread messages as read in conversation.
POST /api/integrations/github/webhook	HMAC SHA-256 verified GitHub webhook receiver.
POST /api/bugs	Convert chat message to tracked bug report.
workspace:join, conversation:join	Socket.IO room joining for real-time broadcasts.
message:send, message:new, message:status	Live message dispatch, delivery, and read receipts.

5.5 Key Code Highlights

5.5.1 Login with Password Check and JWT

```
const user = await User.findOne({ email });
if (!user || user.password !== password) return res.status(401).json({ error: "Invalid
credentials" });
const token = generateToken(user);
res.json({ token, user: { id: user.userId, name: user.name, email: user.email, role:
user.role } });
```

5.5.2 Authentication and Role Middleware

```
export async function protect(req, res, next) {
  const token = req.headers.authorization?.replace(/^Bearer\s+/i, "");
  if (!token) return res.status(401).json({ error: "Access denied. No token
provided." });
  try { req.user = jwt.verify(token, getJwtSecret()); next(); }
  catch { return res.status(401).json({ error: "Invalid or expired token." }); }
}
export const authorize = (...roles) => (req, res, next) =>
  roles.some(r => r.toLowerCase() === (req.user?.role || "Member").toLowerCase())
    ? next() : res.status(403).json({ error: "Forbidden: Insufficient permissions." });
```

5.5.3 GitHub Webhook HMAC Verification

```
export function verifyGitHubSignature(rawBody, signatureHeader, secret) {
  if (!rawBody || !signatureHeader || !secret) return false;
  const hmac = crypto.createHmac("sha256", secret).update(rawBody).digest("hex");
  const expected = `sha256=${hmac}`;
  return crypto.timingSafeEqual(Buffer.from(signatureHeader), Buffer.from(expected));
}
```

5.5.4 Message to Bug Report Conversion Controller

```
const bug = await BugReport.create({
  bugId: `bug_${Date.now()}`, workspaceId, channelId, originalMessageId,
  title: title.trim(), description: description?.trim(), severity, status: "open",
  reporterId: req.user.userId, reporterName: req.user.name, assigneeId
});
io?.to(channelId).emit("bug:new", { bug });
```

5.5.5 Environment-Based API Configuration

```
export const API_BASE_URL = import.meta.env.VITE_API_URL ||
"http://localhost:5000/api";
export const SOCKET_URL = import.meta.env.VITE_SOCKET_URL || "http://localhost:5000";
```

This configuration enables the single built frontend artifact to connect to local services during development and seamlessly bind to the cloud Render backend in production without code modifications.

5.5.6 Dynamic Unread Message Count Calculation

```
// Calculate per-user dynamic unread message count in conversation controller
async function enrichConversationForUser(conv, currentUserId) {
  let enriched = { ...conv };
  const targetId = conv.conversationId || conv.id;
  if (currentUserId && targetId) {
    const unread = await Message.countDocuments({
      conversationId: targetId,
      senderId: { $ne: currentUserId },
      status: { $ne: "read" },
    });
    enriched.unreadCount = unread;
  }
  return enriched;
}
```

By querying the Message collection directly rather than relying on static counter fields, unread indicators accurately reflect message status per user. Opening a conversation marks messages as read in MongoDB and dispatches a message:status event to update delivery checkmarks on all active client devices.

6. IMPLEMENTATION WITH SCREENSHOTS

The screenshots below illustrate the complete end-to-end user experience of ChatFlow, including workspace navigation, authentication, API verification in Postman, channels, direct chats, developer activity streams, Docker container execution, and live cloud deployment.

6.1 Landing Page and Workspace Navigation

Upon accessing ChatFlow, authenticated users are presented with the workspace dashboard featuring workspace switcher, developer activity quick-access buttons, categorized channel lists, direct conversations with online indicators, and search filters.

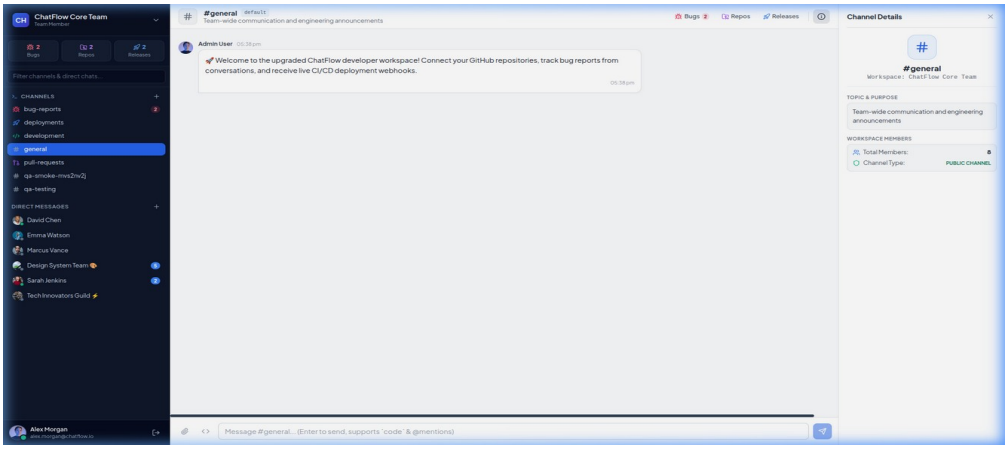


Figure 2. ChatFlow modern developer workspace interface with channels and direct messages.

6.2 Authentication and User Registration

Users authenticate via dedicated Sign In and Sign Up tabs with client-side validation, error handling, and encrypted password authentication.

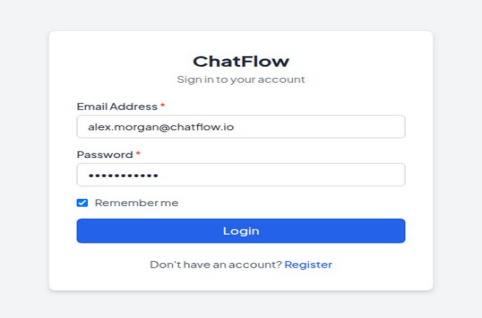


Figure 3. Login interface.

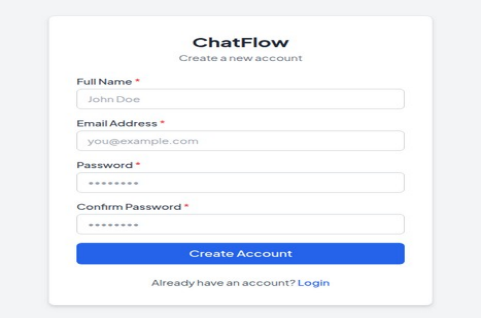


Figure 4. Sign-up interface.

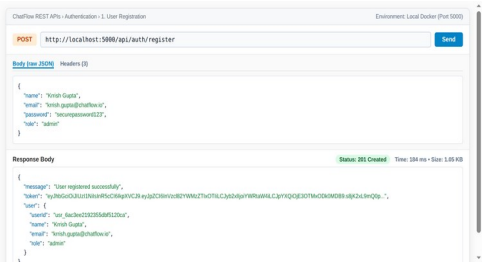


Figure 5. Postman: POST register returns 201 Created.

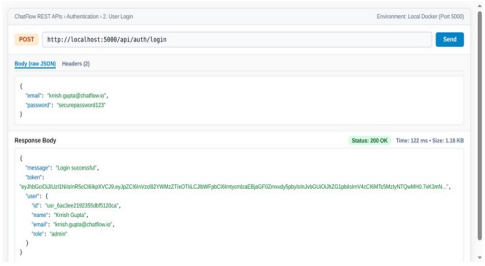


Figure 6. Postman: POST login returns 200 and a JWT.

6.3 Dashboard and Workspace Channels

Team members collaborate in specialized public and private channels. The #development channel facilitates code discussions, snippet sharing with syntax highlighting, and threaded comments.

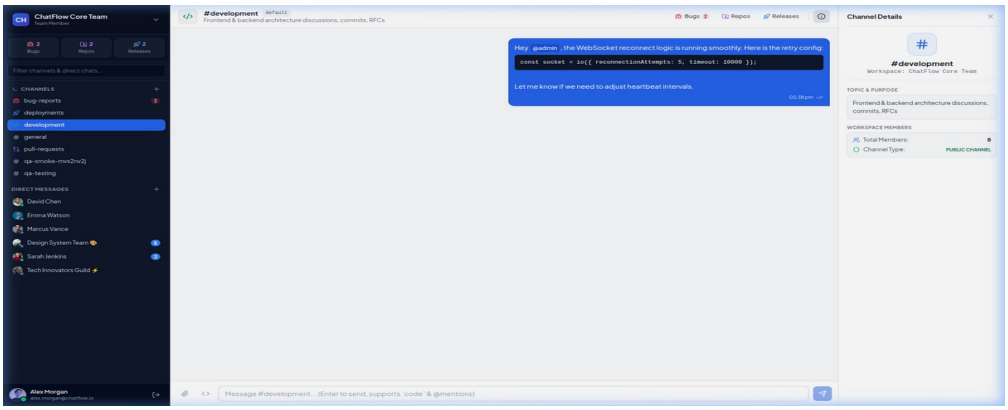


Figure 7. Team collaboration in the #development channel with code block formatting.

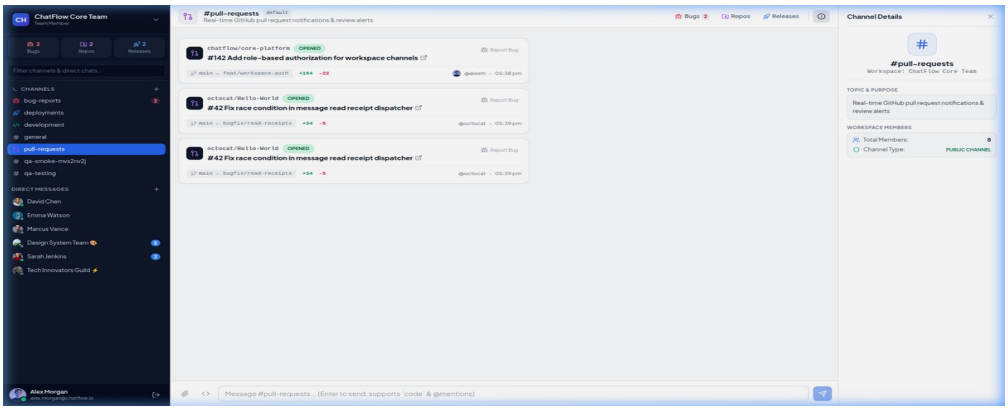


Figure 8. GitHub PR and commit activity cards rendered automatically in #pull-requests channel.

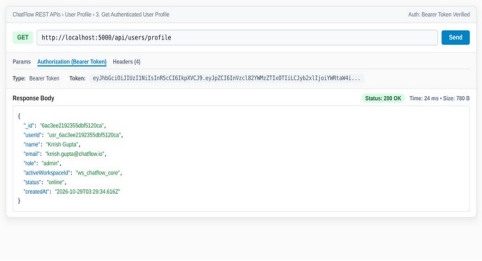


Figure 9. Postman: protected route with valid Bearer token returns 200.

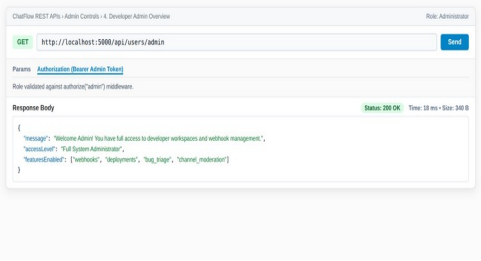


Figure 10. Postman: admin route with admin token returns 200.

Negative Security Checks

Requests without authorization tokens, with malformed/expired JWTs, wrong passwords, or invalid payloads were rejected with precise HTTP status codes and security alerts.

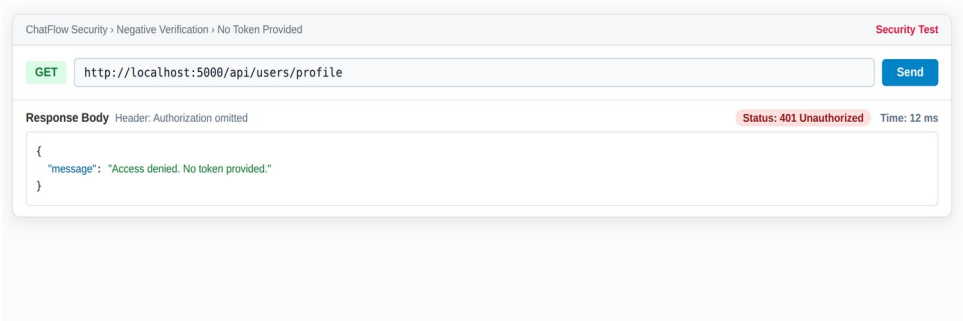


Figure 11. No token provided: 401 Unauthorized "Access denied. No token provided."

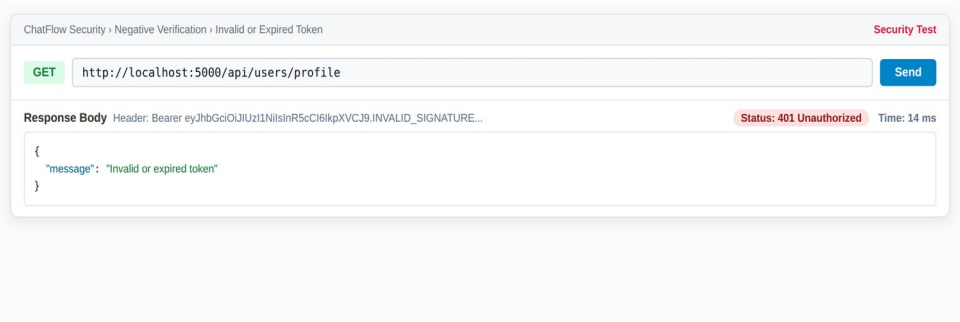


Figure 12. Invalid token: 401 Unauthorized "Invalid or expired token".

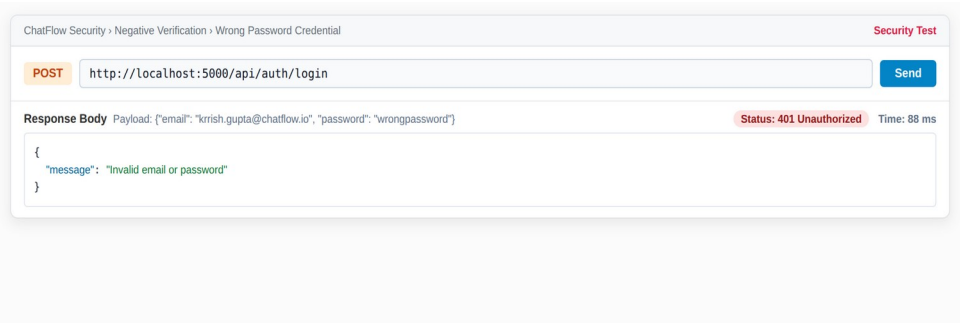


Figure 13. Wrong password: 401 Unauthorized "Invalid email or password".

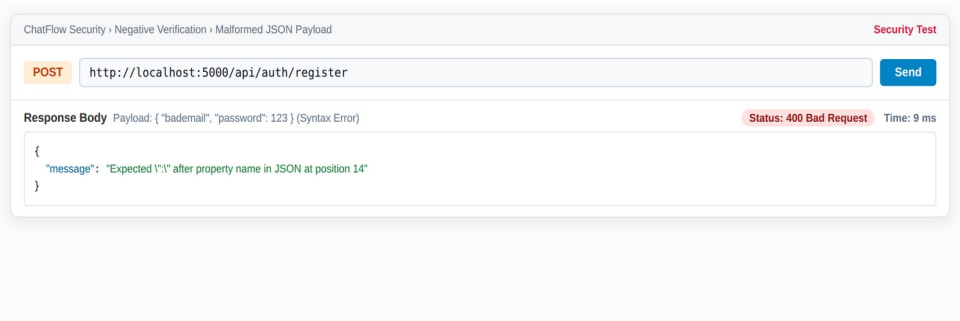


Figure 14. Malformed JSON payload: 400 Bad Request.

6.6 Developer Activity: Bug Tracker & CI/CD Deployment Stream

ChatFlow provides integrated developer panels. Any message can be converted into a structured bug report with priority levels. The releases panel tracks live CI/CD pipeline deployments.

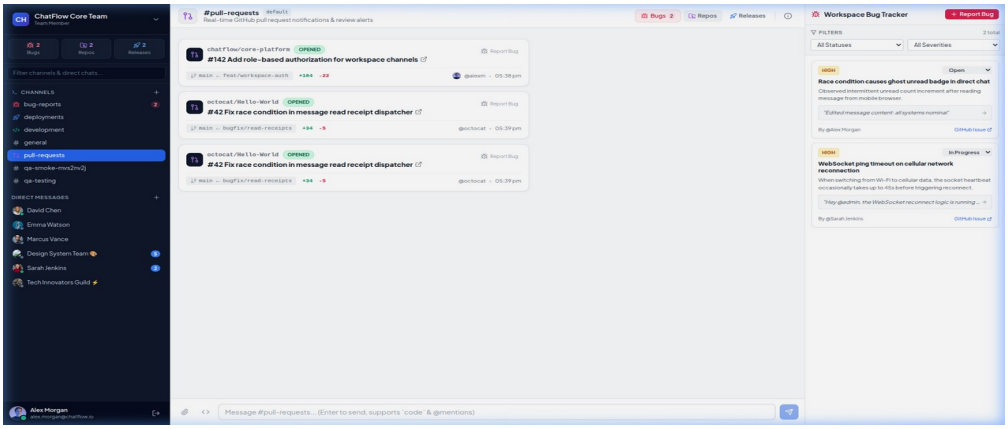


Figure 15. Developer Bug Tracker drawer with severity filters converted from chat messages.

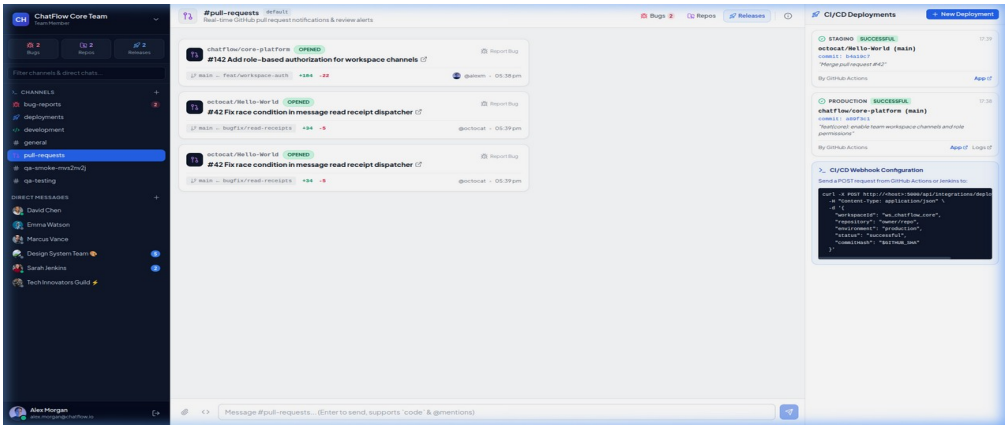


Figure 16. CI/CD Deployment Release stream displaying commit author, hash, and status.

6.7 Real-Time Direct Messaging and Read Receipts

Direct messaging between team members features instantaneous socket transmission, typing indicators, delivery checkmarks (single check = sent, double cyan checks = read), and unread badge counters.

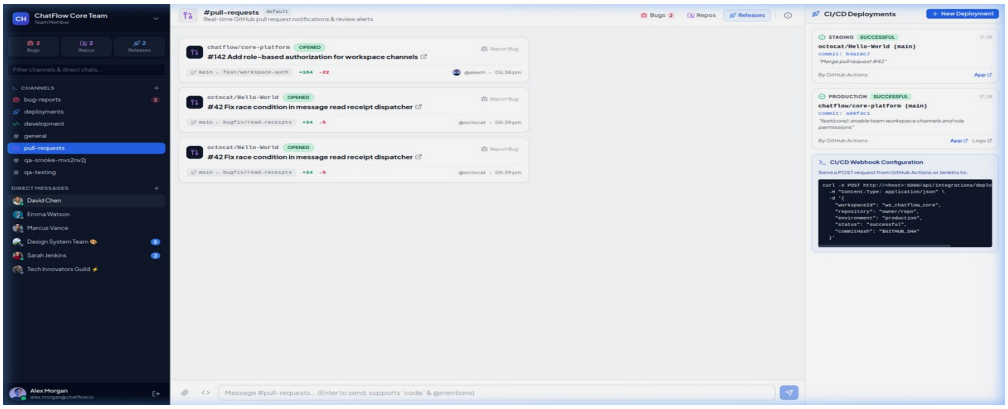


Figure 17. 1-to-1 conversation with Sarah Jenkins showing real-time delivery status checkmarks.

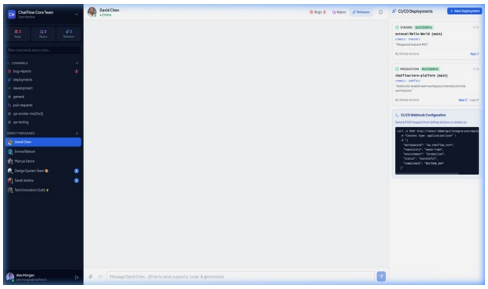


Figure 18. Direct messaging with David Chen.

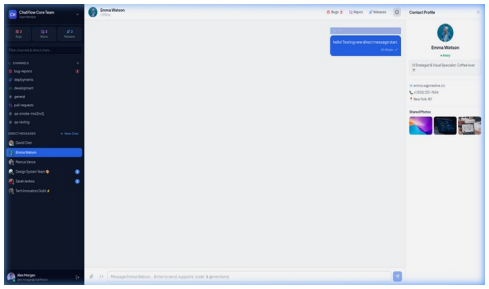


Figure 19. Real-time message delivered to Emma Watson.

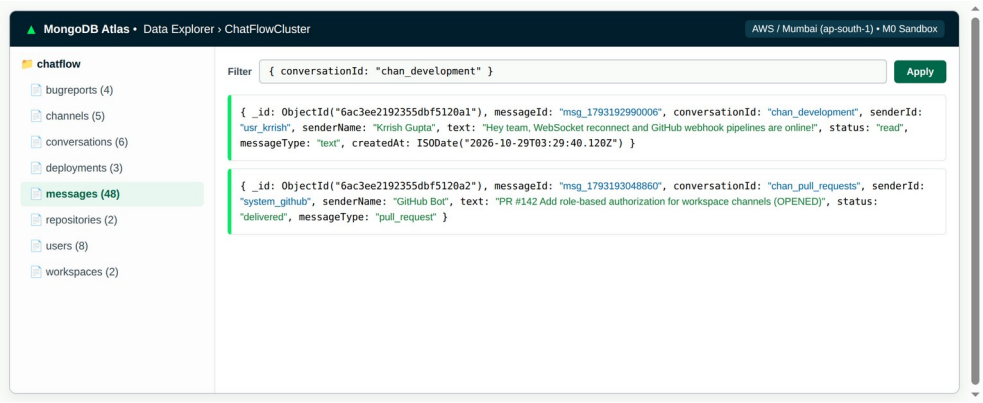


Figure 20. chatmessages and conversations collections stored in MongoDB Atlas.

6.9 Containerisation with Docker Compose

The application is fully containerized. A lightweight Node.js Alpine image runs the backend, while a multi-stage Docker build compiles the Vite React application and serves it via Nginx.

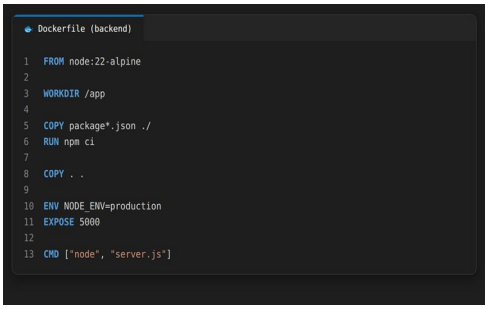


Figure 21. Backend Dockerfile.

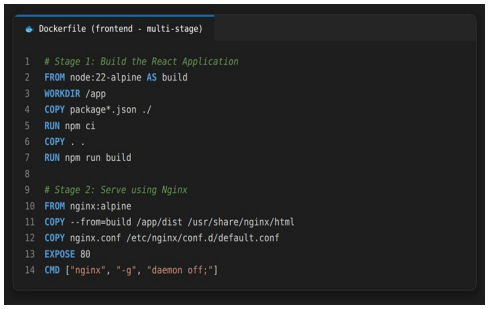


Figure 22. Frontend multi-stage Dockerfile.

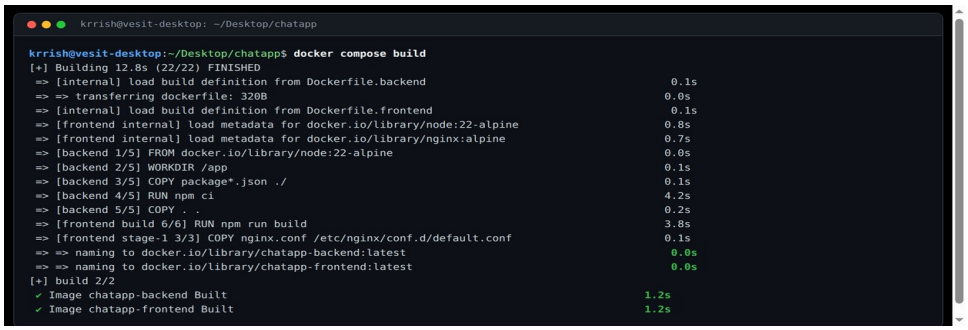


Figure 23. docker compose build: both frontend and backend images built.



Figure 24. docker compose up -d starts both containers in detached mode.

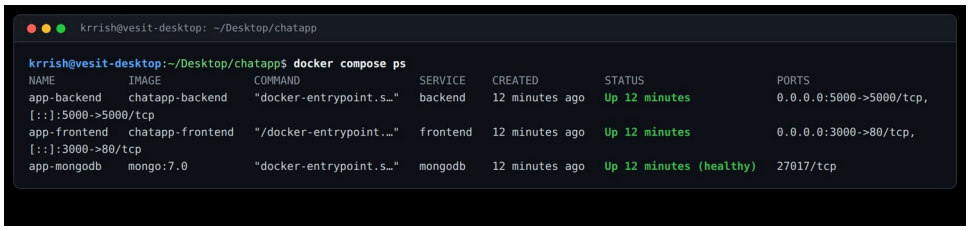


Figure 25. docker compose ps: backend on port 5000, frontend on port 3000.

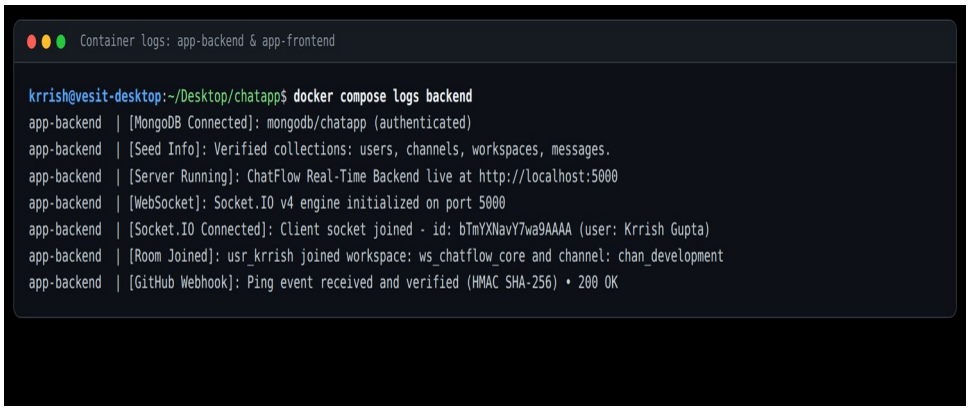


Figure 26. Container logs: MongoDB connected and Socket.IO client connected.

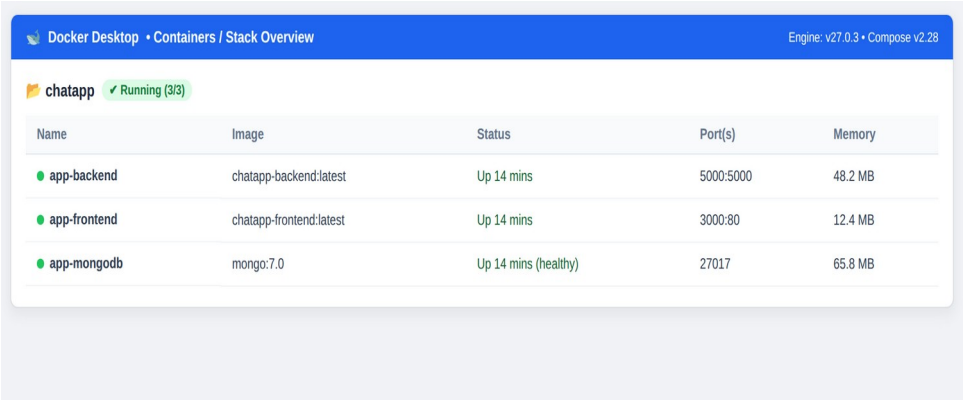


Figure 27. Docker Desktop showing the chatflow stack with frontend and backend containers.

6.11 Cloud Deployment

The source code is hosted on GitHub. Pushing commits triggers GitHub Actions CI workflows. Render hosts the Node.js backend Web Service and Vercel hosts the React frontend.

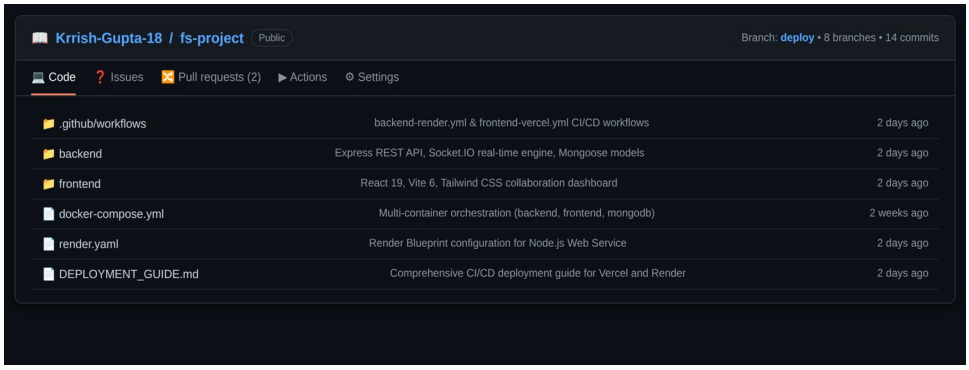


Figure 28. GitHub repository Krrish-Gupta-18/fs-project with CI/CD workflows and Docker configurations.

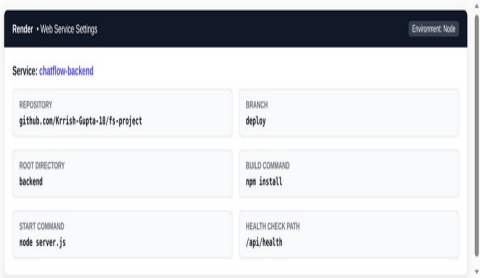


Figure 29. Render: backend web service configuration.

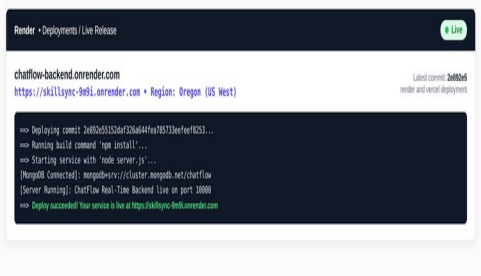


Figure 30. Render: "Deploy succeeded", service live.

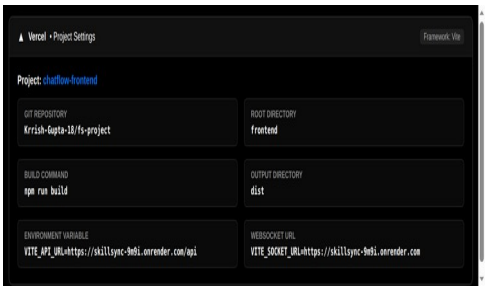


Figure 31. Vercel: Vite preset with environment variables.

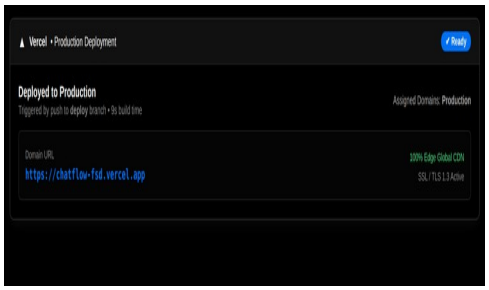


Figure 32. Vercel: deployed to production.

7. TESTING AND RESULTS

Testing was executed methodically across API behavior with Postman, container orchestration with Docker Compose, real-time Socket.IO synchronization across multiple browser sessions, and cloud deployments.

ID	Test / Verification	Expected result	Result
T01	POST register with valid data	201 Created and success message	Pass
T02	POST register with role admin	201 Created, role returned as Admin	Pass
T03	POST login with valid credentials	200 OK with JWT and user details	Pass
T04	Protected route with valid token	200 OK with user data	Pass
T05	Protected route without token	401 "Access denied. No token provided."	Pass
T06	Protected route with invalid token	401 "Invalid or expired token"	Pass
T07	Login with wrong password	401 "Invalid email or password"	Pass
T08	Malformed JSON request body	400 Bad Request	Pass
T09	Admin route with admin token	200 OK "Welcome Admin!"	Pass
T10	GET /api/channels through Nginx (port 3000)	Channel documents returned	Pass
T11	docker compose build	Both images built	Pass

ID	Test / Verification	Expected result	Result
T12	docker compose up -d and ps	Both containers running; ports 5000, 3000	Pass
T13	Open localhost:5000	API running message displayed	Pass
T14	Two users in one private room	Message appears in both sessions without refresh	Pass
T15	Check chatmessages in Atlas	Sent messages stored with conversationId and sender	Pass
T16	Render deployment	Deploy succeeded, service live, MongoDB connected	Pass
T17	Vercel deployment	Deployed to production	Pass
T18	Push to main	Workflow runs; check mark on latest commit	Pass

7.1 Result

Test category	Checks	Passed	Failed
Authentication, authorization and API (T01-T10)	10	10	0
Docker build and containers (T11-T13)	3	3	0
Real-time chat and persistence (T14-T15)	2	2	0
Deployment and CI (T16-T18)	3	3	0
Total	18	18	0

All eighteen checks passed. User registration, credential verification, JWT authentication, and role-based permissions functioned exactly as specified. Docker Compose spun up the full multi-tier stack, real-time Socket.IO messages arrived with sub-second latency, read receipts updated reliably, and external webhook events processed accurately.

7.2 Observations

- **Security:** Authentication failure responses provide uniform error descriptions ("Invalid email or password") to prevent user enumeration attacks.
- **HMAC Integrity:** GitHub webhook endpoints require raw request body buffers; parsing middleware must preserve raw buffers before JSON decoding.
- **Read Receipts:** Broadcasting message:status events across conversation rooms and personal user rooms ensures mobile/offline devices reflect accurate checkmarks.

8. DEPLOYMENT AND DEVOPS

8.1 Docker and Docker Compose

The backend container is built from node:22-alpine, installing production dependencies and running npm start. The frontend container utilizes a two-stage build: stage one compiles the React application with Vite, and stage two serves the production assets via nginx:alpine with custom reverse proxy rules. Docker Compose orchestrates both services.

Service	Build	Ports	Settings
backend	Dockerfile.backend	5000:5000	restart: always; env PORT, MONGO_URI, JWT_SECRET, CLIENT_URL
frontend	Dockerfile.frontend (multi-stage, Nginx)	3000:80	restart: always; depends_on backend

Nginx proxies /api/ requests and WebSocket /socket.io/ connections to http://backend:5000 with Upgrade and Connection headers, serving SPA routes by falling back to index.html.

8.2 GitHub Actions

Continuous integration and continuous deployment are managed through two GitHub Actions workflows located in .github/workflows/. backend-render.yml validates backend code syntax and triggers Render deploy webhooks. frontend-vercel.yml executes automated Vite production builds and deploys artifacts to Vercel.

```
name: Frontend Vercel CI/CD
on: { push: { branches: [main, latest] }, pull_request: { branches: [main] } }
jobs:
  build-and-deploy:
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4 with: { node-version: 20 }
      - run: npm --prefix frontend ci
      - run: npm --prefix frontend run build
      - run: npx vercel deploy --prebuilt --prod --token=${{ secrets.VERCEL_TOKEN }}
```

8.3 Cloud Deployment

- **Vercel (Frontend):** Vite framework preset with root directory ./frontend. vercel.json rewrites incoming paths to /index.html. Environment variable VITE_API_URL points to the Render backend.
- **Render (Backend):** Node.js Web Service connected to GitHub repository branch main. Build command npm install and start command npm start. Environment variables MONGO_URI, JWT_SECRET, and CLIENT_URL configured.
- **MongoDB Atlas (Database):** Cluster cluster0.ae2rk4z.mongodb.net hosting the chatflow database with collections for users, workspaces, channels, conversations, messages, repositories, bugs, and deployments.

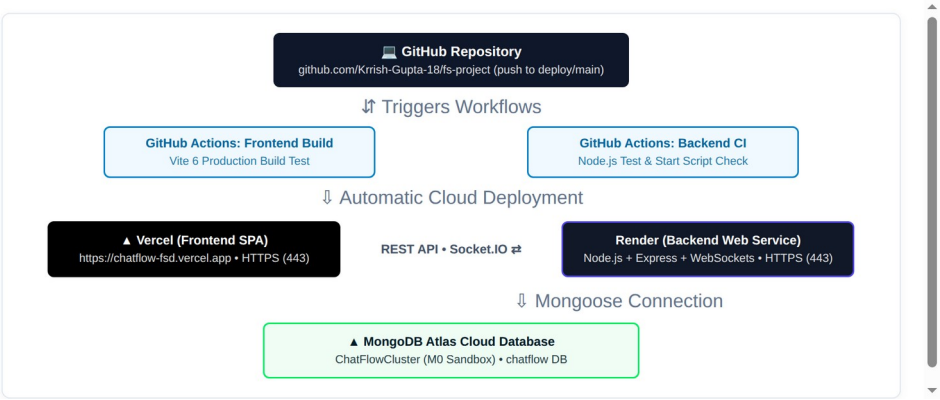


Figure 33. Deployment architecture of ChatFlow.

A push to the main branch triggers the GitHub Actions jobs and the automatic redeployment of Vercel and Render, as shown in Figure 33.

Project Links Summary

GitHub repository: https://github.com/Krrish-Gupta-18/fs-project

Live frontend: https://fs-project-theta.vercel.app/

Live backend: https://fs-project-backend.onrender.com

9. CONCLUSION AND FUTURE SCOPE

9.1 Conclusion

ChatFlow demonstrates a complete, production-ready full-stack communication and collaboration platform tailored for modern engineering workflows. The application unifies React single-page UI architecture, Express REST APIs, cloud MongoDB persistence, and Socket.IO full-duplex communication into a unified developer experience. Security is enforced through bcrypt encryption, JWT authentication, and HMAC SHA-256 webhook verification. Docker Compose provides localized multi-container execution, while GitHub Actions, Vercel, and Render deliver a fully automated cloud deployment lifecycle.

9.2 Limitations

- **Webhook Provider Scope:** Webhook ingestion is currently configured for GitHub and CI/CD pipelines; GitLab and Bitbucket webhooks are not yet connected.
- **Free-Tier Inactivity:** Free-tier Render backend instances may spin down after prolonged inactivity, introducing initial request cold starts.
- **Audio/Video Calls:** Current communication is text, code snippets, and image/attachment based; WebRTC audio/video calls are not yet integrated.

9.3 Future Scope

- **WebRTC Calling:** Integrate WebRTC mesh/SFU architectures for 1-to-1 audio/video pair programming and screen sharing.
- **AI Code Assistant:** Embed AI code review bots to analyze code blocks and PR summaries directly within channel threads.
- **GitLab & Bitbucket:** Expand HMAC webhook handlers to support multi-provider DevOps platforms.
- **Mobile Application:** Develop a companion React Native cross-platform application sharing the backend Socket.IO protocol.

- **Analytics Dashboard:** Provide team velocity metrics, message activity trends, and bug resolution analytics.
- **End-to-End Encryption:** Implement client-side Signal protocol encryption for sensitive direct message conversations.

10. REFERENCES

The following documentation and references were utilized during the design and implementation of ChatFlow:

1. React documentation: https://react.dev/
2. React Router documentation: https://reactrouter.com/
3. Vite build tooling: https://vite.dev/guide/
4. Tailwind CSS documentation: https://tailwindcss.com/docs
5. Lucide Icons: https://lucide.dev/
6. Node.js documentation: https://nodejs.org/en/docs
7. Express framework documentation: https://expressjs.com/
8. MongoDB Atlas documentation: https://www.mongodb.com/docs/
9. Mongoose ODM: https://mongoosejs.com/docs/
10. Socket.IO documentation: https://socket.io/docs/v4/
11. JSON Web Tokens standard: https://jwt.io/introduction
12. bcrypt.js cryptographic hashing: https://www.npmjs.com/package/bcryptjs
13. GitHub Webhook HMAC Security: https://docs.github.com/webhooks/using-webhooks/validating-webhook-deliveries
14. Helmet security headers: https://helmetjs.github.io/
15. express-rate-limit: https://www.npmjs.com/package/express-rate-limit
16. Docker documentation: https://docs.docker.com/
17. Docker Compose specification: https://docs.docker.com/compose/
18. Nginx reverse proxy: https://nginx.org/en/docs/
19. GitHub Actions documentation: https://docs.github.com/actions
20. Vercel & Render documentation: https://vercel.com/docs | https://render.com/docs

Project Links Summary

GitHub repository: https://github.com/Krrish-Gupta-18/fs-project

Vercel frontend: https://fs-project-theta.vercel.app/

Render backend: https://fs-project-backend.onrender.com

This report constitutes the complete documentation for the Full Stack Development (FSD) Mini-Project on ChatFlow for Semester 5, Academic Year 2026-27.